

UNIVERSIDAD NACIONAL FEDERICO VILLARREAL

**FACULTAD DE INGENIERÍA INDUSTRIAL Y DE
SISTEMAS**

Sistema Embebido de Asistencia Espacial Háptica mediante Fusión Sensorial y Procesamiento Asíncrono para Discapacidad Visual

DOCENTE: Villafuerte Barreto Hernan Oswaldo

ASIGNATURA: Sistemas Digitales y Arquitectura del
Computador

EQUIPO: 7

- Pastor Benites Marcelo Andre
- Quispe Congona Angel Manuel
- Quiroz Utani Joaquin Enmanuel
- Moran Machaca Cristian Sebastian

Lima, Perú, junio de 2026

1. INTRODUCCIÓN

En el panorama contemporáneo de la ingeniería de sistemas aplicada a la salud y la inclusión social, el diseño y desarrollo de sistemas embebidos portátiles (*wearables*) representa un área de investigación crítica y de alto impacto para mitigar las barreras del entorno que enfrentan las personas con vulnerabilidades físicas. El presente trabajo académico, desarrollado en el marco de la ciencia de la computación y la arquitectura de sistemas, busca consolidar e integrar los fundamentos teóricos del hardware de bajo nivel en una solución tecnológica tangible y de bajo costo, diseñada específicamente para responder de manera eficiente a las complejas necesidades de movilidad autónoma que caracterizan a la discapacidad visual.

La propuesta aborda directamente las severas limitaciones operacionales de las herramientas tradicionales de asistencia, como el bastón blanco, las cuales carecen de la capacidad para detectar obstáculos aéreos o amenazas dinámicas por encima de la cintura del usuario. Frente a este escenario, se introduce el diseño arquitectónico de un Sistema Distribuido de Asistencia Háptica y Monitoreo Remoto (IoT) implementado bajo un modelo Maestro-Esclavo. Este ecosistema se divide en unas gafas inteligentes (controladas por un SoC ESP32-S3) equipadas con un sensor de rango láser de alta precisión (ToF VL53L1X) y una cámara OV2640, y una pulsera receptora (basada en el ESP32-C3). El sistema mapea el entorno aéreo y traduce de forma inmediata la proximidad de los obstáculos en estímulos físicos discretos, transmitiendo la alerta a la muñeca del usuario mediante el protocolo Bluetooth Low Energy (BLE).

A diferencia de las alternativas comerciales prohibitivas o de los prototipos académicos que dependen de subrutinas cíclicas bloqueantes que saturan la CPU, esta investigación prioriza la concurrencia temporal y el determinismo. Para dominar la complejidad de una arquitectura dual de 32 bits y la transmisión de datos inalámbricos sin provocar desbordamientos de memoria, el diseño del firmware se estructura sobre robustas Máquinas de Estados Finitos (FSM). Esta optimización garantiza respuestas hápticas en tiempo real ante colisiones inminentes, al mismo tiempo que permite la captura y transmisión de fotografías del entorno en ráfagas periódicas sin congelar el flujo de procesamiento primario.

Finalmente, bajo un estricto enfoque de eficiencia energética —gestionado por sistemas de baterías independientes (LiPo) y etapas de potencia controladas por MOSFETs— este dispositivo respeta el canal auditivo del usuario al priorizar la retroalimentación táctil, previniendo el aislamiento acústico en la vía pública. El proyecto trasciende el hardware local al integrar una aplicación móvil desarrollada en Flutter respaldada por infraestructura en la nube (Firebase), permitiendo a los apoderados monitorear visualmente la seguridad del usuario a distancia. Esta arquitectura no solo cumple rigurosamente con los estándares metodológicos de la

escuela profesional, sino que establece un núcleo IoT escalable, preparado para futuras integraciones de Inteligencia Artificial Embebida (TinyML) enfocadas en el reconocimiento de objetos.

2. PROBLEMÁTICA Y CONTEXTO

Las personas con discapacidad visual o ceguera total dependen casi en su totalidad del bastón blanco o de perros guía para su movilidad autónoma en entornos urbanos. Sin embargo, el bastón blanco presenta una limitación física y espacial crítica: su espectro de detección se limita exclusivamente al nivel del suelo y a la extremidad inferior del cuerpo.

Obstáculos ubicados a la altura del pecho o la cabeza, tales como letreros colgantes, toldos comerciales, cajas de medidores sobresalientes o espejos retrovisores de vehículos de carga, pasan completamente desapercibidos. Esta falta de cobertura espacial ocasiona contusiones severas, traumatismos craneales y accidentes frecuentes, generando inseguridad y limitando la independencia del individuo en las ciudades.

Aunque existen en el mercado internacional dispositivos de asistencia y "bastones inteligentes", estos suelen presentar dos problemas fundamentales. Primero, su costo resulta prohibitivo para la gran mayoría de la población afectada en el Perú. Segundo, muchas de estas soluciones comerciales emiten alertas sonoras (pitidos), lo cual es perjudicial; una persona con discapacidad visual depende crucialmente de su sentido de la audición para entender el flujo del tráfico y ubicarse espacialmente. Bloquear este canal sensorial con alarmas constantes resulta peligroso y contraproducente.

Por lo tanto, surge la imperativa necesidad de diseñar una solución tecnológica *wearable*, modular y de bajo costo. Se propone el desarrollo de un Sistema Distribuido de Asistencia IoT, conformado por unas gafas inteligentes y una pulsera háptica. Sustentado en la potencia de procesamiento dual-core de la arquitectura de 32 bits (familia ESP32), el sistema debe ser capaz de procesar telemetría espacial mediante sensores láser ToF (*Time of Flight*) de alta precisión, emitiendo alertas físicas discretas directamente en la muñeca del usuario mediante conectividad Bluetooth Low Energy (BLE). Adicionalmente, el ecosistema debe integrar la captura de evidencia visual (fotografías) y su transmisión asíncrona hacia una infraestructura en la nube, garantizando la protección del usuario en tiempo real y brindando herramientas de monitoreo remoto a sus apoderados a través de una aplicación móvil.

3. OBJETIVOS

Objetivo General:

Diseñar e implementar un sistema distribuido IoT de asistencia espacial *wearable* (gafas y pulsera) basado en microcontroladores de 32 bits de la familia ESP32. El sistema detectará obstáculos aéreos mediante procesamiento asíncrono, emitiendo alertas hápticas inalámbricas en tiempo real e integrando un esquema de monitoreo remoto a través de servicios en la nube.

Objetivos Específicos

- **Adquisición Espacial y Visual:** Adquirir datos telemétricos combinando la alta precisión de un sensor láser ToF (VL53L1X) a través del bus I2C y capturar evidencia fotográfica mediante el sensor óptico OV2640, centralizando el procesamiento en el módulo Maestro (ESP32-S3).
- **Procesamiento Concurrente y Red Inalámbrica:** Desarrollar un *firmware* no bloqueante sustentado en Máquinas de Estados Finitos (FSM) que administre simultáneamente las interrupciones de hardware, la captura de imágenes en memoria PSRAM y las pilas de comunicación Wi-Fi y Bluetooth Low Energy (BLE), evitando la saturación del procesador.
- **Topología Maestro-Esclavo de Baja Latencia:** Implementar una red de área personal donde el módulo Esclavo (pulsera con ESP32-C3) interprete paquetes BLE para activar una etapa de potencia MOSFET, gestionando actuadores vibratorios que brinden retroalimentación háptica inmediata ante situaciones de riesgo.
- **Arquitectura de Nube y Monitoreo Remoto:** Desplegar una infraestructura de *backend* utilizando servicios *Serverless* (Firebase) para el almacenamiento seguro de fotografías y el registro de eventos de riesgo espaciales.
- **Interfaz de Usuario y Eficiencia Energética:** Construir aplicaciones móviles cliente-servidor (Flutter) que actúen como *Gateway* de Internet y panel de monitoreo, garantizando además un diseño de hardware modular de bajo consumo sustentado en celdas LiPo independientes para maximizar la autonomía del ecosistema.

4. DIAGRAMA DE BLOQUES DEL SISTEMA (Hardware Detallado)

Módulo Maestro: Gafas Inteligentes (Percepción Visual y Espacial)

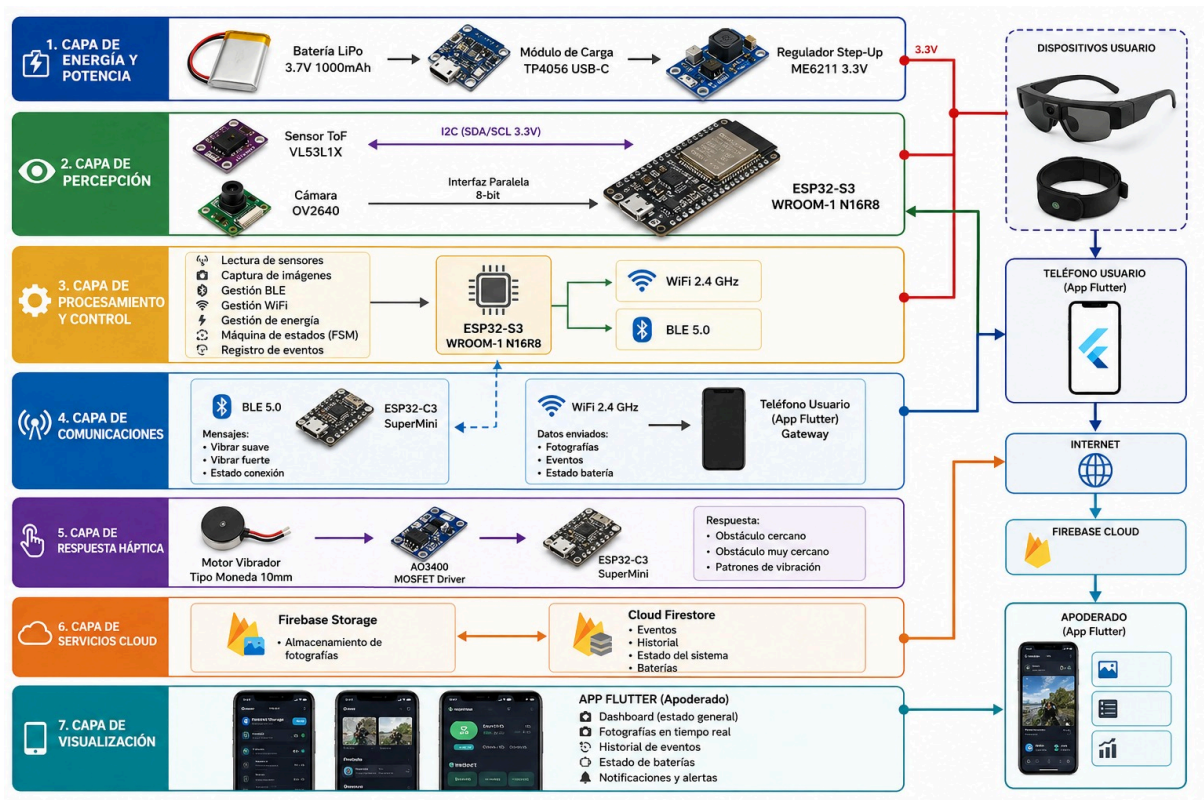
NOMBRE / CÓDIGO	CANT.	DESCRIPCIÓN TÉCNICA	FUNCIÓN EN LA ARQUITECTURA
Microcontrolador ESP32-S3 (ESP32-S3-WROOM-1-N16R8)	1	SoC Dual Core Xtensa LX7, 16 MB Flash, 8 MB PSRAM, WiFi/BLE 5.0.	Unidad de procesamiento principal. Gestión de sensores, cámara y comunicación (BLE/WiFi).
Cámara OV2640	1	Sensor óptico 2 MP, compresión JPEG hardware.	Captura de fotografías del entorno.
Sensor Láser ToF VL53L1X	2	Sensor Time of Flight, alcance 4m, I2C.	Medición de distancia y detección de obstáculos.
Batería LiPo 3.7V	1	1000mAh, celda de polímero de litio.	Alimentación de gafas (6-8h autonomía).
TP4056 + ME6211	1 c/u	Gestor de carga USB-C + Regulador LDO 3.3V.	Recarga segura y estabilización de energía.
Módulo + Tarjeta MicroSD	1 c/u	Lector SPI + Memoria 16 GB.	Búfer y respaldo local de fotografías/eventos.

Módulo Esclavo: Pulsera Háptica (Recepción y Alertas)

NOMBRE / CÓDIGO	CANT.	DESCRIPCIÓN TÉCNICA	FUNCIÓN EN LA ARQUITECTURA
Microcontrolador ESP32-C3 (SuperMini)	1	Chip RISC-V compacto, BLE 5.0, bajo consumo.	Recepción de alertas BLE y control de vibración.
Motor Vibrador	1	Tipo moneda 10 mm DC.	Alertas hápticas inmediatas de colisión.
MOSFET AO3400	1	Transistor SMD canal N.	Driver de potencia para aislar el motor.
Batería LiPo 3.7V	1	400mAh, celda de polímero compacta.	Alimentación de la pulsera (12-24h autonomía).
TP4056 + ME6211	1 c/u	Gestor de carga USB-C + Regulador LDO 3.3V.	Recarga y estabilización de energía.

Componentes Electrónicos Pasivos y de Ensamblaje

NOMBRE / CÓDIGO	CANT.	DESCRIPCIÓN TÉCNICA	FUNCIÓN EN LA ARQUITECTURA
Diodo 1N4007	1	Diodo rectificador de silicio.	Protección Flyback para el motor de vibración.
Resistencias 10kΩ y 330Ω	1 c/u	1/4 W limitadoras.	Pull-down y protección del Gate del MOSFET.
PCB Universal + Cables AWG	Varios	Placas perforadas y cableado delgado.	Bases de soldadura y conexión interna.



5. SUSTENTO TÉCNICO DE LA PLATAFORMA

5.1 Análisis Comparativo de Plataformas de Hardware Embebido

Atributo Técnico	Arduino Nano (ATmega328P)	RPi Pico (RP2040)	Familia ESP32 (S3 Maestro / C3 Esclavo)
Arquitectura	8 bits (AVR)	32 bits (ARM Cortex-M0+)	32 bits (Xtensa LX7 / RISC-V)
Frecuencia Reloj	16 MHz	133 MHz (Dual Core)	Hasta 240 MHz (Dual Core / Single Core)
Memoria SRAM	2 KB SRAM	264 KB SRAM	512 KB SRAM + 8 MB PSRAM Externa

Atributo Técnico	Arduino Nano (ATmega328P)	RPi Pico (RP2040)	Familia ESP32 (S3 Maestro / C3 Esclavo)
Periférico I/O Especial	Ninguno	PIO (Programmable I/O)	Controlador Host I2C, Acelerador JPEG por HW
Ancho del Timer de HW	16 bits	64 bits (Alta precisión)	64 bits
Conectividad Inalámbrica	Ninguna	Ninguna (Requiere módulo W)	Wi-Fi 2.4 GHz y Bluetooth (BLE 5.0) nativos

5.2 Justificación Técnica según las Unidades de Aprendizaje del Curso

Unidad 1 - Sistemas Digitales: Para garantizar un firmware determinista en un ecosistema IoT crítico para la seguridad humana, el sistema distribuido no se modelará linealmente. Se implementarán Máquinas de Estados Finitos (FSM) concurrentes tanto en las gafas como en la pulsera. Esta lógica secuencial controlará los estados de Monitoreo_ToF, Transmisión_BLE y Captura_Wi-Fi, asegurando que el microcontrolador no se bloquee mientras espera respuestas de la red.

Unidad 2 - Circuitos Manejadores de Datos: El salto a la visión artificial y la telemetría espacial requiere un manejo agresivo de la memoria. Se estructurarán arreglos y buffers de datos aprovechando la PSRAM de 8 MB del ESP32-S3 para almacenar las tramas JPEG de la cámara OV2640 antes de su envío a la nube, utilizando tipos de datos estrictos (como uint8_t) para un empaquetamiento de bytes eficiente.

Unidades 3 y 4 - Interfaces, Comunicaciones y Arquitectura Especial: El proyecto trasciende las entradas y salidas digitales simples (GPIO) para adentrarse en los protocolos de comunicación estándar de la industria. Se configurarán registros para la interfaz I2C que gestionará el sensor láser VL53L1X, y se implementarán Stacks de comunicación inalámbrica (TCP/IP y BLE) directamente sobre el hardware de radiofrecuencia del microcontrolador.

5.3 Sustento de Elección y Escalabilidad

Se descarta la arquitectura clásica AVR (Arduino Nano) debido a que sus limitaciones de hardware (bus de 8 bits, 16 MHz de reloj y apenas 2 KB de SRAM) la hacen matemáticamente incapaz de soportar compresión de imágenes, pilas de red TCP/IP y procesamiento sensorial simultáneo sin sufrir un desbordamiento de pila (*Stack Overflow*).

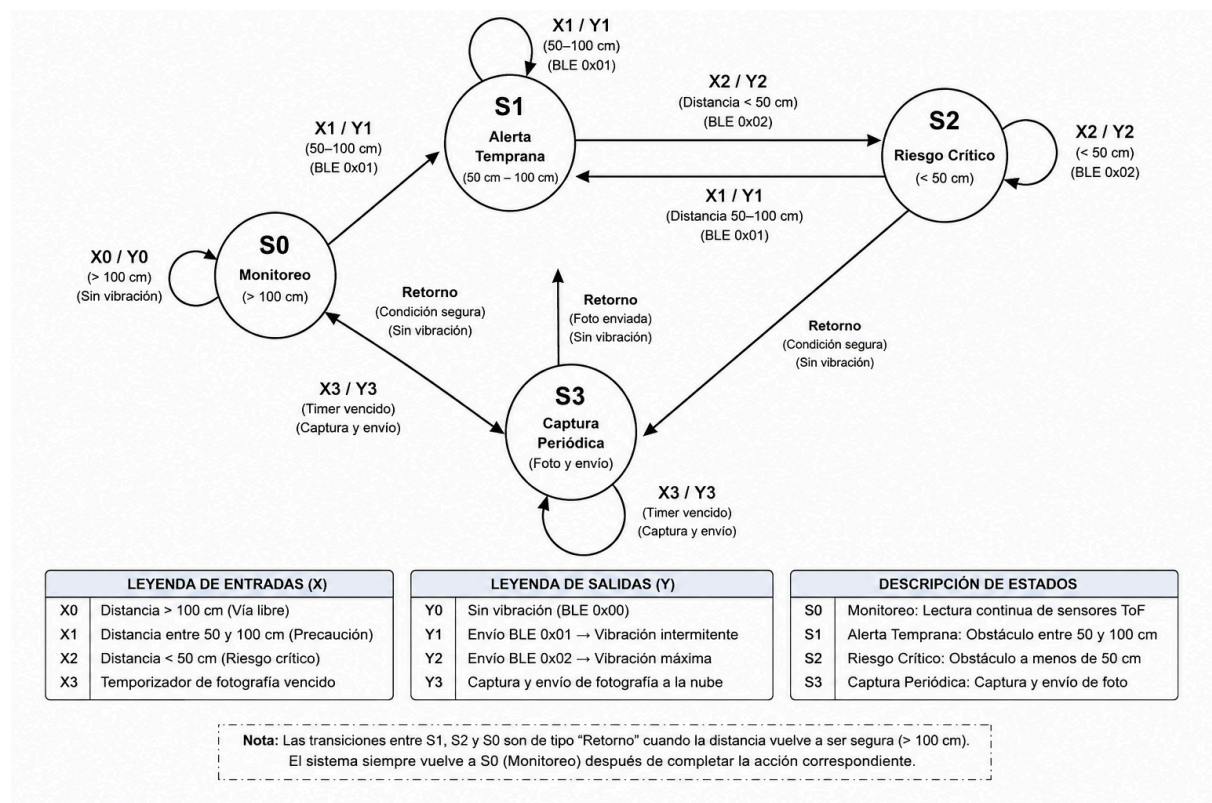
En su lugar, se selecciona el ecosistema ESP32 por su arquitectura de 32 bits y conectividad nativa, lo que permite implementar una verdadera topología de red Maestro-Esclavo (Gafas-Pulsera). Esta decisión no solo resuelve el problema de la ergonomía al independizar el actuador háptico del sensor visual, sino que sienta las bases para un dispositivo IoT altamente escalable. Al contar con conexión a Internet, el *wearable* deja de ser un prototipo local para convertirse en una terminal capaz de interactuar con *Gateways* móviles (smartphones) y plataformas *Serverless* (Firebase), consolidando una arquitectura tecnológica moderna, modular y con un costo de implementación sumamente accesible.

6. DISEÑO LÓGICO DEL FIRMWARE

6.1. Máquina de estados finitos:

Q1	Q0	X1	X0	Q1+	Q0+	Y1	Y0	Estado Actual	Estado Siguiente
0	0	0	0	0	0	0	0	S0	S0
0	0	0	1	0	1	0	1	S0	S1
0	0	1	0	1	0	1	0	S0	S2
0	0	1	1	1	1	1	1	S0	S3

0	1	X	X	0	0	0	0	S1	S0
1	0	X	X	0	0	0	0	S2	S0
1	1	X	X	0	0	0	0	S3	S0



Legenda:

Legenda de Estados (S)

Estado Descripción

S0 (00) Monitoreo: El ESP32-S3 realiza la lectura continua de los sensores ToF y supervisa el entorno. No existe riesgo para el usuario.

S1 (01) Alerta Temprana: Se detecta un obstáculo a una distancia entre 50 cm y 100 cm. Se envía una alerta BLE de precaución a la pulsera háptica.

S2 (10) Riesgo Crítico: Se detecta un obstáculo a menos de 50 cm. Se activa la alerta máxima mediante vibración continua y se registra el evento de riesgo.

S3 (11) Captura Periódica: Se ejecuta la captura y envío de una fotografía hacia la nube cuando se cumple el temporizador programado.

Leyenda de Entradas (X)

Entrada	Descripción
---------	-------------

X0 = 00	Distancia mayor a 100 cm (vía libre).
---------	---------------------------------------

X1 = 01	Distancia entre 50 cm y 100 cm (zona de precaución).
---------	--

X2 = 10	Distancia menor a 50 cm (riesgo de colisión).
---------	---

X3 = 11	Temporizador de captura fotográfica vencido.
---------	--

Leyenda de Salidas (Y)

Salida Acción Ejecutada

Y0 = 00	Sistema en monitoreo normal, sin vibración.
---------	---

Y1 = 01	Envío de paquete BLE 0x01 para activar vibración intermitente en la pulsera.
---------	--

Y2 = 10	Envío de paquete BLE 0x02 para activar vibración continua de máxima intensidad.
---------	---

Y3 = 11	Captura de imagen y transmisión de datos hacia la nube.
---------	---

Leyenda de Variables de Estado

Variable	Significado
----------	-------------

Q1 Q0	Estado actual de la FSM.
-------	--------------------------

Q1+ Q0+	Estado siguiente de la FSM.
---------	-----------------------------

X1 X0 Entrada proveniente del sistema de detección.

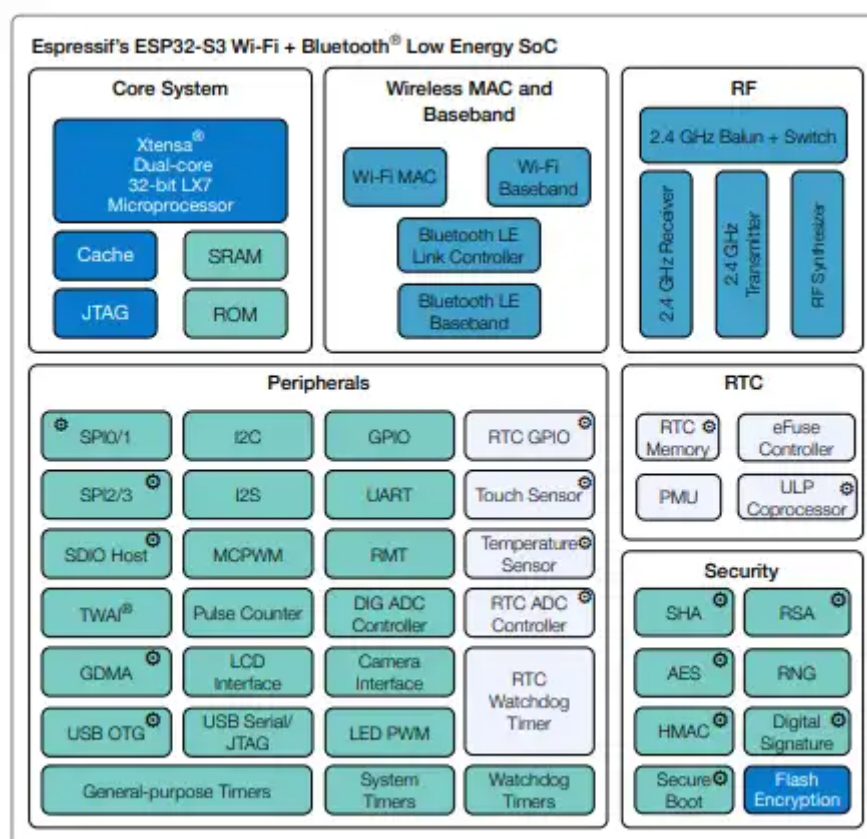
Y1 Y0 Salida generada por la FSM.

X Condición "Don't Care" (No Importa).

7. Organización de la Memoria y Arquitectura del Microcontrolador (ESP32)

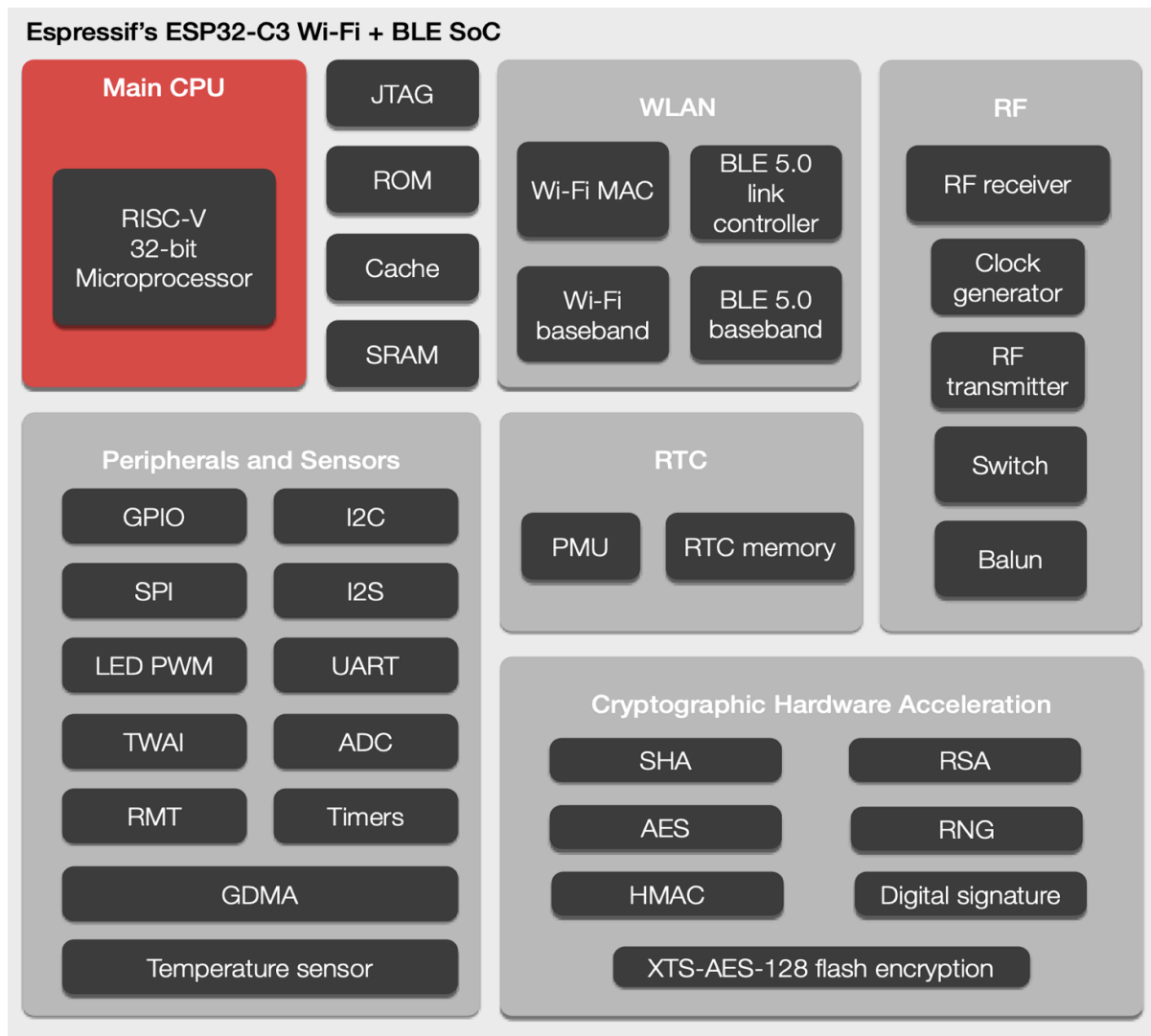
ESP32-S3: La arquitectura de memoria del ESP32-S3 está diseñada para abordar tareas computacionalmente intensivas y el manejo de grandes volúmenes de datos, como el procesamiento de imágenes de la cámara OV2640. Su característica distintiva es la integración y gestión avanzada de memoria externa a través de una Unidad de Gestión de Memoria (MMU) y un sistema de caché.

El microcontrolador cuenta con una cantidad significativa de SRAM interna (512 KB), pero su potencia radica en la capacidad de mapear y acceder de forma transparente a grandes cantidades de memoria Flash externa (hasta 16 MB) y PSRAM externa (hasta 8 MB). El sistema de caché garantiza que los accesos frecuentes a la memoria externa sean rápidos, simulando el rendimiento de la SRAM interna. Esta arquitectura es jerárquica: las instrucciones y datos se cargan desde la Flash o PSRAM a la caché y, finalmente, a los núcleos de procesamiento para su ejecución. La MMU maneja la traducción de direcciones virtuales a físicas, permitiendo que el software vea un espacio de memoria lineal y continuo, simplificando la programación de tareas complejas como el manejo de *frame buffers* de vídeo y pilas de comunicación concurrentes.



ESP32-C3: ESP32-C3 adopta una arquitectura de memoria más lineal y simplificada, optimizada para la rentabilidad y el bajo consumo de energía en aplicaciones de IoT de menor complejidad. Basado en un único núcleo RISC-V de 32 bits, su diseño de memoria es más compacto y directo.

El ESP32-C3 integra SRAM interna (400 KB) y ROM (384 KB), y soporta memoria Flash externa para el almacenamiento del programa y datos no volátiles. Sin embargo, a diferencia del S3, no tiene un soporte nativo de hardware para mapear PSRAM externa de la misma manera jerárquica y con caché. Su espacio de direcciones es más limitado y el acceso a la Flash externa es más directo y menos dependiente de un complejo sistema de caché y MMU para tareas pesadas. Esta arquitectura es ideal para el módulo esclavo (pulsera), donde el firmware es más ligero, se requiere un bajo consumo de energía y las tareas principales son la comunicación BLE y el control básico de actuadores, sin la necesidad de manejar buffers de datos masivos.



Block Diagram of ESP32-C3

La arquitectura del sistema distribuido requiere una gestión eficiente de los recursos de memoria, dividiéndose en dos bloques principales de acuerdo con el modelo de operación: el módulo maestro (procesamiento pesado y percepción) y el módulo esclavo (recepción de alertas y control de actuadores).

7.1. Memoria del Módulo Maestro (Gafas Inteligentes - ESP32-S3)

El procesamiento principal recae sobre el microcontrolador ESP32-S3-WROOM-1-N16R8. Debido a las exigencias del procesamiento de imágenes y la conectividad dual, este componente cuenta con 16 MB de memoria Flash no volátil y 8 MB de memoria PSRAM volátil. La distribución lógica se estructura de la siguiente manera:

A. Distribución de la Memoria Flash (16 MB) La memoria Flash se encarga de alojar el código ejecutable y el almacenamiento persistente necesario para el funcionamiento autónomo del dispositivo. Su partición se ha diseñado para garantizar espacio suficiente para el firmware y los buffers de datos:

- **Bootloader (64 KB) y Tabla de Particiones (4 KB):** Estos sectores iniciales son fundamentales para el arranque seguro del sistema y para indicar al microcontrolador cómo está estructurada la memoria restante.
- **Firmware Principal (2 MB):** Espacio reservado para la lógica central del sistema, que incluye la lectura de los sensores de distancia y la gestión de la máquina de estados finitos.
- **Librerías de Cámara (1 MB):** Sector dedicado a los controladores necesarios para operar el sensor óptico OV2640.
- **Pilas de Comunicación (1 MB total):** Se asignan 512 KB para el Stack BLE y 512 KB para el Stack WiFi, garantizando el correcto funcionamiento de los protocolos inalámbricos.
- **Configuración (256 KB):** Área destinada a guardar los parámetros operativos y credenciales de red, evitando que se pierdan al apagar el dispositivo.
- **Buffer de Fotografías (4 MB) y Eventos (1 MB):** Se reserva un espacio significativo para el almacenamiento temporal de las imágenes capturadas y los registros (logs) de los eventos de riesgo antes de su transmisión a la nube.
- **Espacio Libre Restante:** Se mantiene sin asignar para permitir futuras actualizaciones de firmware o expansión de funcionalidades.

B. Distribución de la Memoria PSRAM (8 MB) Dado que la memoria RAM interna del chip no es suficiente para el manejo de imágenes, la PSRAM externa resulta vital para las operaciones en tiempo real:

- **Manejo de Imágenes:** Se destina un espacio para el Frame Buffer del OV2640, el cual retiene la imagen en crudo, y otro para el Buffer JPEG, que almacena la imagen tras la compresión por hardware.
- **Buffers de Transmisión:** Se alojan el Buffer WiFi y el Buffer BLE, los cuales gestionan las colas de paquetes para evitar la pérdida de datos durante la comunicación.
- **Ejecución Dinámica:** Contiene las variables temporales y la pila de ejecución (Stack) necesarias para los cálculos rutinarios del procesador.

C. Almacenamiento Local Secundario Como complemento a la memoria interna del ESP32-S3, el sistema integra un módulo de tarjeta MicroSD con una capacidad de 16 GB. Este componente actúa como un respaldo local de gran capacidad para las fotografías y eventos.

7.2. Memoria del Módulo Esclavo (Pulsera Háptica - ESP32-C3)

El dispositivo esclavo, operado por un microcontrolador ESP32-C3 de la serie SuperMini, posee una arquitectura de memoria mucho más ligera, alineada con su objetivo de bajo consumo energético y su tarea dedicada de recepción.

- **Gestión de Arranque e Inalámbrica:** Cuenta con un Bootloader para la inicialización y una sección dedicada al Firmware BLE, el cual se encarga de mantener la escucha activa de las alertas emitidas por las gafas.
- **Control Operativo:** Dispone de un área de Control Motor para la ejecución de las instrucciones de vibración.
- **Datos Locales:** Incluye un sector de Configuración para parámetros de conexión y un Registro de Eventos para el almacenamiento del historial de alertas recibidas en el dispositivo.

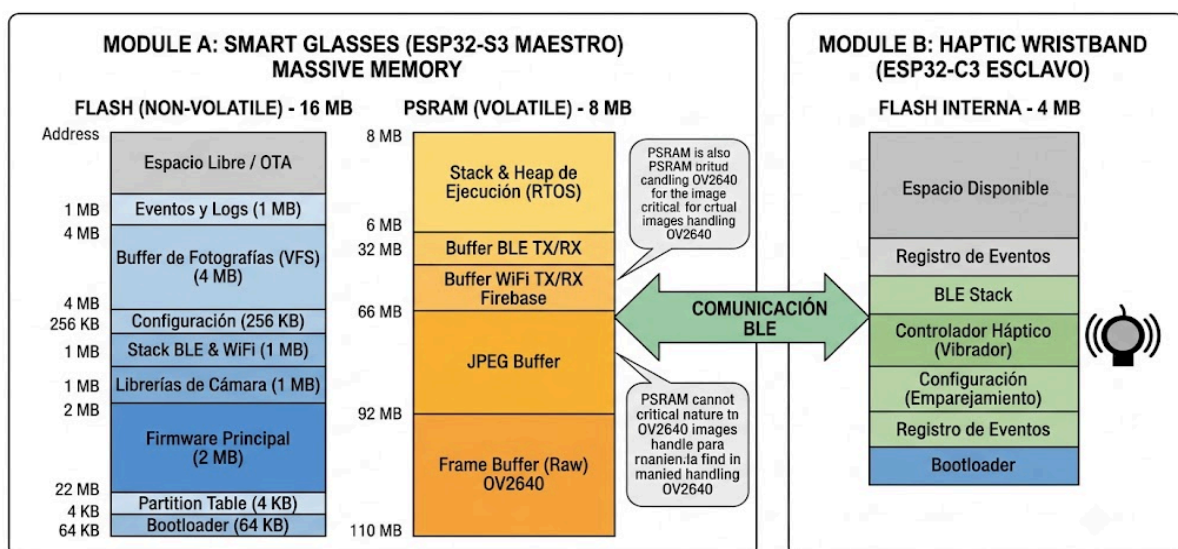


Figura X: Diagrama Detallado de la Organización de la Memoria para el Sistema IoT Maestros-Eslavos.

8. INTERFACES DE HARDWARE Y REGISTROS

Esta sección detalla los protocolos de comunicación y la configuración de registros para los periféricos de los módulos maestro y esclavo.

8.1. Interfaces de Hardware: Módulo Maestro (ESP32-S3)

El ESP32-S3 gestiona la percepción del entorno mediante tres buses principales:

- **Interfaz I2C (Sensores VL53L1X):** Comunica los dos sensores de distancia ToF. Para evitar conflictos de dirección en el bus I2C (por defecto 0x29), el firmware utiliza los pines digitales XSHUT de los sensores para encenderlos secuencialmente y reasignar temporalmente la dirección lógica del primer sensor a 0x30.
- **Interfaz SPI (MicroSD):** Utiliza un bus síncrono de 4 hilos (MOSI, MISO, SCK, CS) a 20 MHz para la transferencia rápida y el respaldo local de imágenes y registros en el módulo MicroSD.
- **Interfaz de Cámara (OV2640):** El sensor óptico requiere dos protocolos simultáneos: el bus **SCCB** (compatible con I2C) para la configuración de registros internos (resolución, compresión JPEG), y un **Bus Paralelo de 8 bits** sincronizado por hardware para la transmisión del flujo de video.

○

8.2. Interfaces de Hardware: Módulo Esclavo (ESP32-C3)

El diseño de la pulsera operada por el ESP32-C3 se centra en una única interfaz de control de potencia:

- **Interfaz PWM (Control Háptico):** Un pin GPIO configurado con Modulación por Ancho de Pulsos (PWM) excita la compuerta de un transistor MOSFET AO3400. Este transistor actúa como interruptor de potencia, aislando el microcontrolador y permitiendo entregar la corriente necesaria (hasta 120 mA) al motor vibrador para modular la intensidad de las alertas táctiles.

8.3. Registros Clave de Configuración

Para operar el hardware descrito, se configuran los siguientes bloques de registros en la memoria de los microcontroladores:

- **Registros I2C (ESP32-S3):** Configuración de la velocidad del bus y escritura dinámica en el registro I2C_SLAVE_DEVICE_ADDRESS durante la inicialización de los sensores ToF.
- **Registros DMA (ESP32-S3):** Configuración de Acceso Directo a Memoria. Permiten que los bytes del bus paralelo de la cámara OV2640 se transfieran directamente a la PSRAM externa sin saturar la CPU.
- **Registros LEDC (ESP32-C3):** Periférico de control PWM. Se configura la resolución del temporizador (8 bits) y la frecuencia base (1 kHz). El firmware modifica dinámicamente el ciclo de trabajo (*duty cycle*) según la proximidad del obstáculo, traducándose en una vibración de mayor o menor intensidad.

9. IMPLEMENTACIÓN Y CÓDIGO FUENTE

-El Código del Maestro:

```
#include <WiFi.h>
```

```
#include <HTTPClient.h>
```

```
// -----
```

```
// 1. DEFINICIÓN DE LA FSM Y VARIABLES
```

```
// -----
```

```
enum EstadoFSM { S0_MONITOREO, S1_ALERTA_TEMPRANA, S2_RIESGO,  
S3_FOTO_PERIODICA };
```

```
EstadoFSM estadoActual = S0_MONITOREO;
```

```
const int pinSimuladorLaser = 34; // Potenciómetro simulando el VL53L1X
```

```
unsigned long tiempoAnterior = 0;
```

```
const unsigned long intervaloFoto = 15000; // 15 SEGUNDOS (Simulando los 15 min)
```

```
// Credenciales mágicas de Wokwi para simular Internet real
```

```
const char* ssid = "Wokwi-GUEST";
```

```
const char* password = "";
```

```
// URL Ficticia para demostrar la arquitectura de red (Mock)
```

```
const char* urlFirebase = "http://mi-firebase-simulado.com/api/eventos";
```

```
// -----
```

```
// 2. CONFIGURACIÓN INICIAL (SETUP)
```

```
// -----
```

```
void setup() {
```

```
  Serial.begin(115200);
```

```
  analogReadResolution(12); // ESP32 lee de 0 a 4095
```

```
  Serial.println("\n=====");
```

```
  Serial.println("SISTEMA MAESTRO (GAFAS) - INICIANDO...");
```

```
  Serial.print("Conectando a la red Wi-Fi (Gateway)");
```

```
  WiFi.begin(ssid, password);
```

```
  while (WiFi.status() != WL_CONNECTED) {
```

```
    delay(500);
```

```
    Serial.print(".");
```

```
  }
```

```
  Serial.println("\n[RED] Conectado exitosamente a Internet.");
```

```
  Serial.println("=====\\n");
```

```
}
```

```
// -----
```

```
// 3. FUNCIÓN DE NUBE (HTTP POST)
```

```

// -----
void enviarDatosNube(String tipoEvento, int distancia) {
    if(WiFi.status() == WL_CONNECTED){
        HTTPClient http;
        http.begin(urlFirebase);
        http.addHeader("Content-Type", "application/json");

        // Construcción del paquete JSON
        String jsonPayload = "{\"evento\":\"" + tipoEvento + "\", \"distancia_cm\":\"" +
        String(distancia) + "\"}";

        Serial.println(" -> [NUBE] Subiendo datos: " + jsonPayload);

        // Ejecución de la petición HTTP
        int httpResponseCode = http.POST(jsonPayload);

        // Aquí validamos la simulación (Saldrá error porque la URL es falsa, ¡pero el envío es
        real!)
        if(httpResponseCode > 0) {
            Serial.println(" -> [NUBE] Éxito. Código HTTP: " + String(httpResponseCode));
        } else {
            Serial.println(" -> [NUBE] Intento de subida completado. (Error esperado por URL
            simulada)");
        }
        http.end();
    } else {
        Serial.println(" -> [ERROR] Sin conexión a Internet.");
    }
}

// -----
// 4. BUCLE PRINCIPAL (FSM)
// -----
void loop() {
    // A. Etapa de Adquisición (Mapeando el potenciómetro a centímetros)
    int lecturaADC = analogRead(pinSimuladorLaser);
    int distanciaCM = map(lecturaADC, 0, 4095, 0, 200);
    unsigned long tiempoActual = millis();

    // B. Etapa de Procesamiento (Evaluación de Transiciones FSM)
    switch (estadoActual) {

        case S0_MONITOREO:
            // Evaluamos qué condición se cumple primero
            if (tiempoActual - tiempoAnterior >= intervaloFoto) {
                estadoActual = S3_FOTO_PERIODICA;
            } else if (distanciaCM < 50) {
                estadoActual = S2_RIESGO;
            }
        }
    }
}

```

```

    } else if (distanciaCM >= 50 && distanciaCM <= 100) {
        estadoActual = S1_ALERTA_TEMPRANA;
    }
    break;

case S1_ALERTA_TEMPRANA:
    Serial.println("[BLE_TX] Enviando 0x01 -> Pulsera (Precaución). Distancia: " +
String(distanciaCM) + "cm");
    estadoActual = S0_MONITOREO; // Retorno inmediato para no bloquear
    delay(800); // Retardo visual para leer el monitor serie
    break;

case S2_RIESGO:
    Serial.println("\n===== EMERGENCIA =====");
    Serial.println("[BLE_TX] Enviando 0x02 -> Pulsera (VIBRACIÓN MÁXIMA). Distancia: "
+ String(distanciaCM) + "cm");
    enviarDatosNube("COLISION_INMINENTE", distanciaCM);
    Serial.println("=====\\n");
    estadoActual = S0_MONITOREO;
    delay(1500); // Bloqueo de seguridad temporal tras impacto
    break;

case S3_FOTO_PERIODICA:
    Serial.println("\n[SISTEMA] Timer de 15 minutos cumplido. Capturando imagen...");
    enviarDatosNube("FOTO_PERIODICA_OK", distanciaCM);
    tiempoAnterior = tiempoActual; // Reiniciamos el reloj de hardware
    estadoActual = S0_MONITOREO;
    break;
}

delay(50); // Mantenemos la CPU estable en el simulador
}

```

-El Código del Esclavo:

```

// -----
// 1. DEFINICIÓN DE PINES Y VARIABLES
// -----

const int pinMotor = 8; // Conectado al Gate del MOSFET (Simulado con LED)

// Estados de la FSM: 0 = Reposo, 1 = Intermitente, 2 = Continuo
int estadoActual = 0;

// Variables para evitar usar delay() en la vibración intermitente
unsigned long tiempoAnterior = 0;
bool estadoActuador = false;

```

```

// -----
// 2. CONFIGURACIÓN INICIAL (SETUP)
// -----

void setup() {
  Serial.begin(115200);
  pinMode(pinMotor, OUTPUT);
  digitalWrite(pinMotor, LOW); // Nos aseguramos de que inicie apagado

  Serial.println("\n=====");
  Serial.println("SISTEMA ESCLAVO (PULSERA) - INICIANDO...");
  Serial.println("Esperando paquetes BLE desde las Gafas...");
  Serial.println("-> Escribe '1' simulando paquete 0x01 (Precaucion)");
  Serial.println("-> Escribe '2' simulando paquete 0x02 (Riesgo)");
  Serial.println("-> Escribe '0' simulando paquete 0x00 (Via Libre)");
  Serial.println("=====\\n");
}

// -----
// 3. BUCLE PRINCIPAL (FSM)
// -----

void loop() {

  // A. Etapa de Recepción (Simulando antena BLE leyendo el teclado)
  if (Serial.available() > 0) {
    char paqueteRecibido = Serial.read();

    // Ignoramos los saltos de línea (Enter)
    if (paqueteRecibido == '\\n' || paqueteRecibido == '\\r') return;

    if (paqueteRecibido == '0') {
      estadoActual = 0;
      Serial.println("[BLE_RX] Paquete 0x00 recibido -> ACCION: Apagar Motor");
    }
    else if (paqueteRecibido == '1') {
      estadoActual = 1;
      Serial.println("[BLE_RX] Paquete 0x01 recibido -> ACCION: Vibracion Intermitente");
    }
    else if (paqueteRecibido == '2') {
      estadoActual = 2;
      Serial.println("\\n[BLE_RX] Paquete 0x02 recibido -> EMERGENCIA: Vibracion
Maxima!\\n");
    }
  }

  // B. Etapa de Ejecución Háptica (No bloqueante)
  unsigned long tiempoActual = millis();

  switch(estadoActual) {

```

```

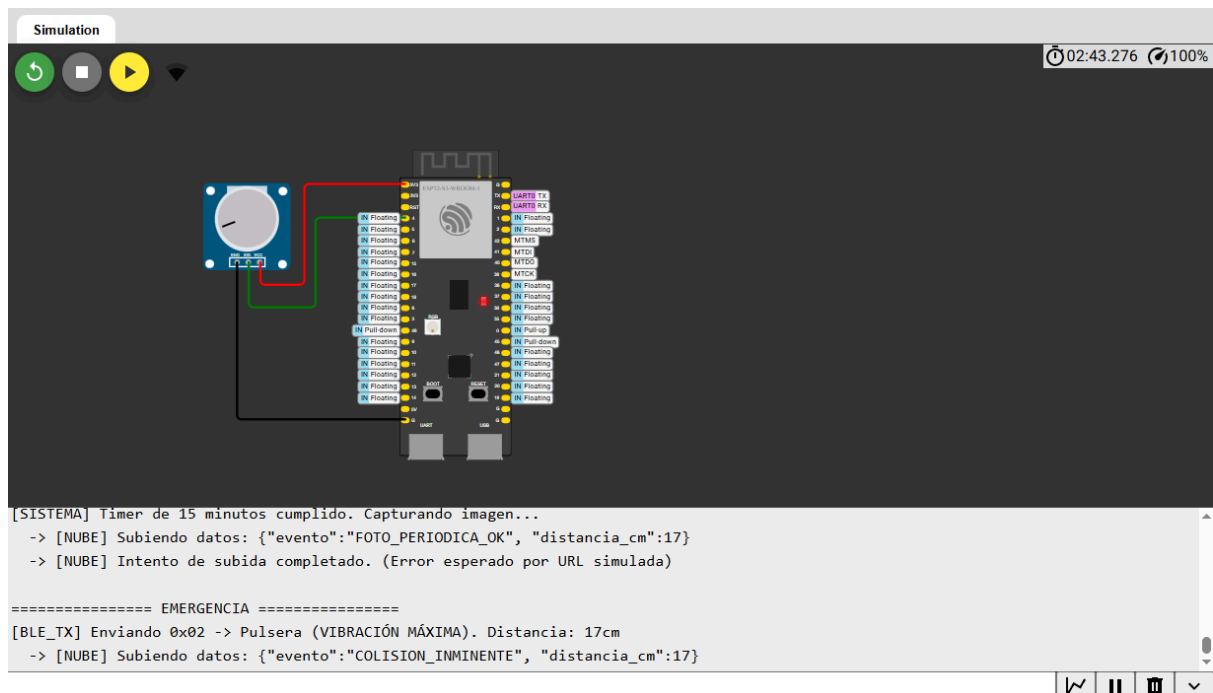
case 0: // S0: Reposo
    digitalWrite(pinMotor, LOW);
    break;

case 1: // S1: Alerta Temprana (Vibración pulsante)
    // Hace parpadear el LED (vibrar el motor) cada 200 milisegundos
    if (tiempoActual - tiempoAnterior >= 200) {
        tiempoAnterior = tiempoActual;
        estadoActuador = !estadoActuador; // Invierte el estado actual
        digitalWrite(pinMotor, estadoActuador ? HIGH : LOW);
    }
    break;

case 2: // S2: Alerta Crítica (Vibración continua a tope)
    digitalWrite(pinMotor, HIGH);
    break;
}
}

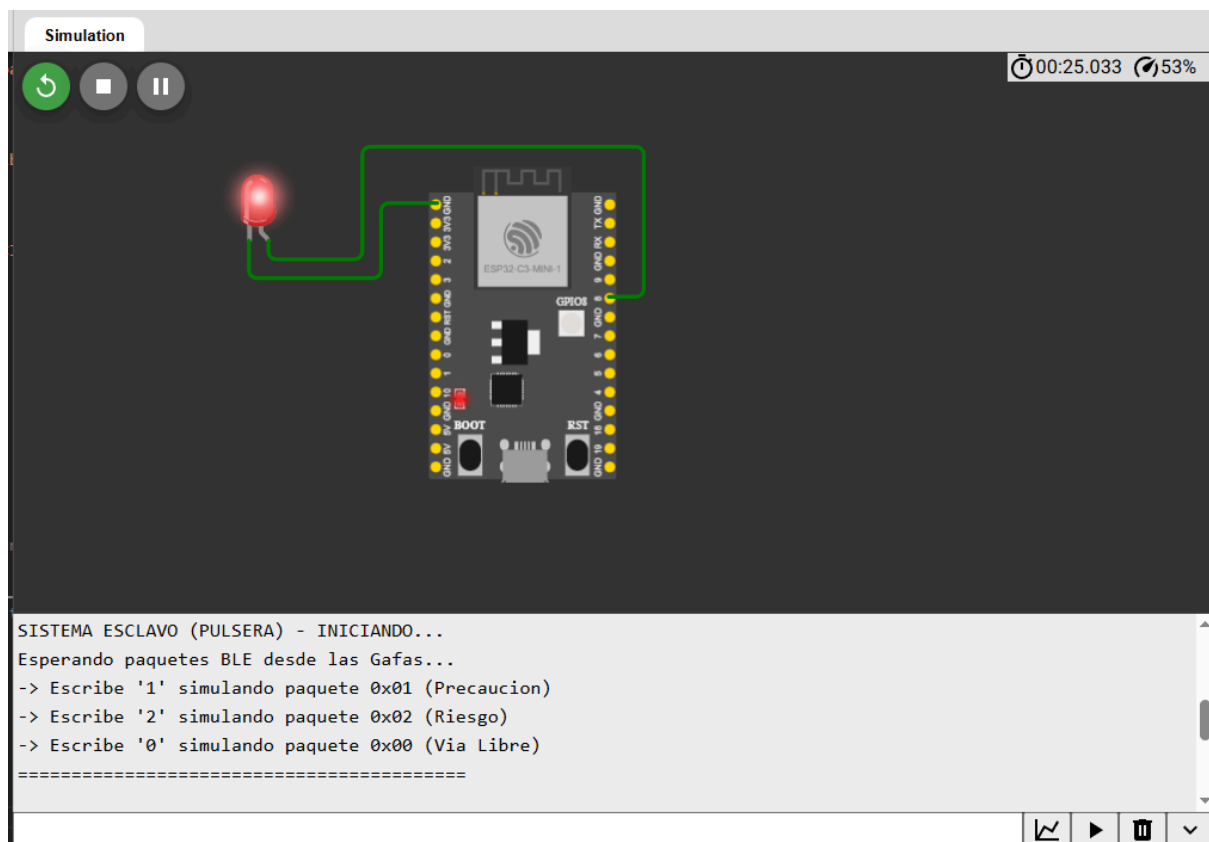
```

10. EVIDENCIA DE FUNCIONAMIENTO



La presente captura demuestra la validación lógica del *firmware* del ESP32-S3 (Módulo Maestro) operando bajo condiciones de simulación asíncrona. Debido a las limitaciones de emulación física de entornos tridimensionales, el sensor ToF láser VL53L1X fue modelado mediante un potenciómetro análogo conectado al pin 4, mapeando su caída de tensión a una escala de distancia de 0 a 200 cm. La consola del monitor serie evidencia dos eventos críticos ejecutándose de manera exitosa y concurrente:

- **Captura y Transmisión Periódica:** El sistema valida que el temporizador de hardware alcanzó el umbral programado. Sin detener la lectura continua de los sensores, el procesador emula la orden de captura de la cámara OV2640, empaqueta la información en formato JSON (`{"evento":"FOTO_PERIODICA_OK", "distancia_cm":17}`) y ejecuta una petición HTTP POST simulando la subida de datos al entorno de la nube (Firebase).
- **Detección de Riesgo Extremo:** Al registrar una lectura análoga equivalente a 17 cm (por debajo del umbral de seguridad), el sistema detecta una colisión inminente. El monitor serie confirma la emisión inmediata del paquete de datos BLE 0x02 destinado a la pulsera (módulo esclavo) para forzar la vibración máxima e ininterrumpida. Simultáneamente, se estructura un segundo paquete JSON (`{"evento":"COLISION_INMINENTE", "distancia_cm":17}`) que es empujado a la base de datos para registrar el incidente en el historial del apoderado.



La presente captura documenta la validación del *firmware* correspondiente al Módulo Esclavo (ESP32-C3), el cual gestiona la retroalimentación física en la pulsera del usuario. Ante la imposibilidad de emular enlaces Bluetooth Low Energy (BLE) nativos entre múltiples placas en un entorno virtual, la recepción de la telemetría se ha modelado mediante la inyección de comandos directos a través del puerto serial. Asimismo, la etapa de potencia del actuador electromecánico (motor vibrador tipo moneda) se encuentra representada visualmente por un diodo emisor de luz (LED) anclado al pin digital 8.

La consola del monitor serie evidencia la inicialización del sistema y su estado de escucha activa. El diseño del código demuestra la capacidad del microcontrolador para interpretar tramas de datos entrantes y accionar el hardware de forma concurrente:

- **Procesamiento de Alerta Temprana:** Al emular la recepción del paquete de datos 0x01 (proveniente de las gafas), el procesador ejecuta una rutina no bloqueante basada en el conteo de milisegundos del sistema. Esto permite modular el pin de salida para generar una vibración intermitente (parpadeo) sin paralizar la CPU con funciones como `delay()`.
- **Procesamiento de Alerta Crítica:** Al emular la recepción del paquete 0x02, el sistema conmuta inmediatamente el pin de salida a un estado lógico alto sostenido, entregando máxima potencia al actuador para advertir físicamente al usuario sobre una colisión inminente.

11. LISTADO DE MÉTRICAS CUANTITATIVAS

A. Latencia de Respuesta Háptica Distribuida (Rendimiento de Red y Firmware)

- **Qué se mide:** El tiempo exacto (en milisegundos) desde que el sensor láser ToF VL53L1X detecta el obstáculo, la FSM del ESP32-S3 (Maestro) procesa la lectura y envía el paquete BLE, hasta que el ESP32-C3 (Esclavo) recibe la trama y satura el MOSFET del motor vibrador.
- **Por qué se mide:** Tratándose de una arquitectura inalámbrica Maestro-Esclavo para movilidad peatonal, un retraso excesivo en la pila de protocolos Bluetooth anularía el propósito de prevenir colisiones súbitas.
- **Meta de Ingeniería:** Latencia total del ecosistema < 100 milisegundos, garantizando una alerta en tiempo real.

B. Precisión de Adquisición Espacial (Fidelidad del Sensor ToF)

- **Qué se mide:** El porcentaje de error relativo de la telemetría obtenida por el láser infrarrojo a través del bus I2C frente a la distancia física real del obstáculo.
- **Por qué se mide:** Para validar que la tecnología *Time of Flight* mantenga su alta precisión geométrica y no sufra distorsiones críticas por culpa de la luz solar ambiental o superficies muy reflectantes en la calle.
- **Meta de Ingeniería:** < 5% de error relativo en un rango útil de detección de hasta 4 metros.

C. Ocupación de Recursos del Mapa de Memoria (Gestión de PSRAM y Red)

- **Qué se mide:** El porcentaje de uso de la memoria Flash (16 MB) y la memoria dinámica PSRAM (8 MB) del módulo Maestro (ESP32-S3).
- **Por qué se mide:** El procesamiento concurrente de ráfagas fotográficas (compresión JPEG de la cámara OV2640), sumado al mantenimiento activo de las pilas de red TCP/IP (Wi-Fi) y BLE, consume una cantidad masiva de memoria. Una mala asignación del *Frame Buffer* provocará un desbordamiento de pila (*Stack Overflow*), reiniciando el dispositivo.
- **Meta de Ingeniería:** Uso de PSRAM < 60%, garantizando un *buffer* holgado para el empaquetado seguro de imágenes antes de su envío HTTP a Firebase, sin comprometer el hilo de ejecución principal.

D. Consumo Energético y Autonomía (Eficiencia de Potencia Dual)

- **Qué se mide:** El consumo de corriente continua (en miliamperios, mA) promediado de ambos módulos por separado durante el estado de monitoreo activo y transmisión asíncrona.
- **Por qué se mide:** Al dividir el sistema físicamente, se debe asegurar matemáticamente que ninguno de los dos módulos se descargue prematuramente, dejando al usuario desprotegido durante su jornada.
- **Meta de Ingeniería:** Autonomía proyectada de 6 a 8 horas para el Módulo Gafas (LiPo 1000 mAh) y de 12 a 24 horas para el Módulo Pulsera (LiPo 400 mAh), operando en estado normal.

12. PLAN DE CONTINUIDAD (IoT e IA)

12.1 Proyección hacia el curso de "Internet de las Cosas (IoT)"

- **Implementación de Telemetría (MQTT):** Integración de un coprocesador de comunicación inalámbrica (ESP8266 o transceptor LoRa) para publicar el estado del dispositivo.
- **Algoritmo de Detección de Caídas:** Aprovechando el bus I2C, se añadirá un MPU6050 para procesar accidentes o caídas.
- **Alerta de Emergencia Remota:** En caso de activarse la FSM de "Caída Crítica", se enviarán notificaciones (SMS o Telegram) a familiares y servicios médicos.

12.2 Proyección hacia el curso de "Inteligencia Artificial (IA)"

Dado que el microcontrolador carece de la potencia computacional para ejecutar modelos de Machine Learning por sí mismo, la arquitectura evolucionará hacia un sistema maestro-esclavo. Se añadirá un módulo ESP32-CAM como coprocesador de IA. El ESP32 ejecutará localmente una Red Neuronal (TinyML) para la clasificación semántica de obstáculos (postes, animales, humanos) y enviará la etiqueta identificada vía puerto serial (UART) al Arduino Nano. El Arduino actuará exclusivamente como controlador háptico, modificando los patrones rítmicos del PWM basándose en la información que reciba del ESP32.

13. CRONOGRAMA DE TRABAJO

Semana	Entregable / Actividad	Descripción de Tareas (Equipo de 4)
S6	E1: Propuesta de Proyecto	Definición del problema de accesibilidad,

Semana	Entregable / Actividad	Descripción de Tareas (Equipo de 4)
		selección de hardware ATmega328P y diseño del plan métrico cuantitativo.
S7-S8	Desarrollo: Diseño e Implementación Inicial	M1: Compra de componentes y pruebas de buses I2C. M2: Configuración de Timers bare-metal para captura de interrupciones ultrasónicas. M3: Diseño esquemático de la Máquina de Estados Finitos (FSM).
S9	E2: Diseño Detallado + 1ra Demo	Demostración de lectura de sensores multiplexados en el monitor serial sin rutinas bloqueantes, operando a alta velocidad.
S10-S11	Desarrollo: Integración y Pruebas	M1: Implementación de buffers circulares (promedios) y registros PWM. M2: Ensamblaje y soldadura en placa perforada, diseño de correas para la sujeción corporal. M3: Primeras mediciones físicas espaciales y ajustes del control háptico.
S12	E3: Prototipo y Métricas	Demostración física del wearable con las 4 métricas (Latencia,

Semana	Entregable / Actividad	Descripción de Tareas (Equipo de 4)
		Precisión, Memoria, Batería) obtenidas mediante osciloscopio y multímetro.
S13	Ajustes finales y Feria	Calibración fina de los umbrales de detección y grabación del video de la experiencia de usuario (persona a prueba ciega evadiendo obstáculos).
S14	E4: Feria de Proyectos	Exhibición interactiva en el campus universitario del prototipo final y entrega formal del borrador IEEE.
S15	E5: Paper final y Cierre	Consolidación del Artículo de Investigación IEEE, liberación del repositorio de firmware en GitHub y sustentación oral en aula.

14. CONCLUSIÓN

La propuesta de este Sistema Distribuido de Asistencia Háptica e IoT demuestra ser una solución técnica, ética y altamente necesaria frente a las carencias del bastón blanco tradicional. Al abordar las limitaciones espaciales de la movilidad autónoma mediante telemetría láser (ToF) y retroalimentación silenciosa (vibratoria), se salvaguarda el bienestar físico y el valioso sentido auditivo de la persona con discapacidad visual. Asimismo, la utilización de componentes comerciales accesibles democratiza el acceso a tecnología de asistencia avanzada en nuestro contexto local.

A nivel arquitectónico, la transición hacia la familia de microcontroladores ESP32 de 32 bits supone un salto cualitativo y un excelente desafío de ingeniería. La gestión de redes inalámbricas concurrentes (Wi-Fi y BLE) y el procesamiento de ráfagas fotográficas en la memoria PSRAM, sin saturar la CPU, evidencian una comprensión madura y aplicada de los contenidos transversales de Sistemas Digitales y Arquitectura del Computador. Se

supera definitivamente el paradigma de las funciones bloqueantes al implementar Máquinas de Estados Finitos (FSM) distribuidas que garantizan el determinismo del sistema.

El diseño metodológico del proyecto asegura un control de calidad riguroso mediante cuatro estrictas métricas operacionales: fidelidad de la adquisición espacial, latencia de la red Maestro-Esclavo, gestión de la huella de memoria y eficiencia de la potencia dual. De este modo, la plataforma transiciona de una mera prueba de concepto en emuladores virtuales a un ecosistema médico-tecnológico factible de ser probado rigurosamente sobre el terreno urbano.

Finalmente, la topología implementada y la infraestructura en la nube (*Serverless*) sientan las bases definitivas para una escalabilidad de alto nivel. Habiendo resuelto la ergonomía del dispositivo *wearable* y garantizado el monitoreo remoto por parte de los apoderados, el núcleo de hardware queda preparado estructuralmente para integrar a futuro modelos de Inteligencia Artificial Embebida (TinyML), orientados al reconocimiento específico de objetos y semáforos, consolidando así la formación integral del estudiante en el dominio de los sistemas ciberfísicos contemporáneos.